

✓ Laboratorio 5: Fine-Tuning de LLMs con LoRA

Curso: **Inteligencia Artificial / PLN**

Autor del Notebook: *Completar por el estudiante*

Fecha de creación: 2025-11-04 17:19

Objetivo: Transformar un modelo de lenguaje base en un asistente especializado usando **LoRA (PEFT)**. Este notebook está diseñado para ejecutarse en **Google Colab** con GPU T4.

1. Objetivos de Aprendizaje

- Comprender la diferencia entre **pre-entrenamiento** y **fine-tuning**.
- Entender **PEFT** y la técnica **LoRA** para reducir memoria.
- Usar el ecosistema de **Hugging Face** (`transformers`, `peft`, `trl`) para afinar un LLM.
- Preparar un **dataset** pequeño para fine-tuning supervisado.
- Ejecutar un proceso de **fine-tuning** en **Colab**.
- Evaluar cualitativamente el cambio de comportamiento **antes** y **después** del fine-tuning.

2. Fundamentos Teóricos (resumen)

Fine-Tuning: Continuar el entrenamiento de un LLM pre-entrenado con un dataset específico para adaptarlo a una tarea/estilo concreto.

Reto de Memoria: Actualizar *todos* los parámetros de un LLM es costoso en VRAM.

LoRA (Low-Rank Adaptation): Congela pesos del modelo y entrena *adaptadores* livianos (<1% del total), reduciendo drásticamente costos de memoria.

Para el detalle completo, consulta la guía del laboratorio.

✓ 3. Prerrequisitos

- Cuenta de Google para **Colab**.
- Conceptos básicos de **Python** y **modelos de lenguaje**.

Configuración de entorno en Colab

1. Abre <https://colab.research.google.com/>
2. Archivo → **Nuevo notebook**
3. Entorno de ejecución → **Cambiar tipo de entorno de ejecución** → **GPU T4**

```
# Verifica la GPU disponible
!nvidia-smi
```

```
Tue May 5 17:11:28 2026
```

```
+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |                       |
+=====+=====+=====+=====+=====+=====+
|  0   Tesla T4               Off        | 00000000:00:04:0 Off |                    0 |
| N/A   40C    P8             9W / 70W   |  0MiB / 15360MiB |      0%      Default |
|                                           | N/A             |                       |
+-----+-----+-----+-----+-----+-----+

```

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	
	ID	ID				Usage	
No running processes found							

4. Instalación de Dependencias

Ejecuta las siguientes celdas para instalar las librerías requeridas.

Nota: La segunda celda actualiza `bitsandbytes` a la versión más reciente. Esto resuelve un `ImportError` de compatibilidad interna que ocurre en Colab cuando se usa la versión fija `0.43.1` junto con las demás librerías.

```
# Instalar librerías con versiones fijas del laboratorio
!pip install -q "transformers==4.40.1" "datasets==2.18.0" "peft==0.10.0" "trl==0.8.6" "bitsandbytes==0.43.1"

# Barra de progreso simulada
138.0/138.0 kB 4.6 MB/s eta 0:00:00
9.0/9.0 MB 69.1 MB/s eta 0:00:00
510.5/510.5 kB 31.4 MB/s eta 0:00:00
199.1/199.1 kB 20.7 MB/s eta 0:00:00
245.2/245.2 kB 12.2 MB/s eta 0:00:00
119.8/119.8 MB 7.0 MB/s eta 0:00:00
302.4/302.4 kB 23.6 MB/s eta 0:00:00
170.9/170.9 kB 9.4 MB/s eta 0:00:00
566.4/566.4 kB 40.5 MB/s eta 0:00:00
3.6/3.6 MB 42.8 MB/s eta 0:00:00
185.2/185.2 kB 13.8 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.2.0 which is incompatible. sentence-transformers 5.4.1 requires transformers<6.0.0,>=4.41.0, but you have transformers 4.40.1 which

```
# FIX: Actualizar bitsandbytes a la versión más reciente.
# Resuelve el ImportError de compatibilidad interna en Colab.
# Ejecutar SIEMPRE después de la celda anterior.
!pip install -q -U bitsandbytes
```

```
60.7/60.7 MB 10.8 MB/s eta 0:00:00
```

5. Cargar el Modelo y Tokenizador (cuantización 4-bit)

Usaremos `microsoft/Phi-3-mini-4k-instruct` en 4-bit (NF4) para caber en la memoria de una T4.

Fix aplicado: Se usa `device_map="auto"` en lugar de `device = torch.device(...)` manual. Esto permite que Hugging Face distribuya automáticamente el modelo entre GPU/CPU, lo que es más robusto y evita errores de dispositivo en Colab.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig

# Config NF4 para cuantización 4-bit
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)

model_id = "microsoft/Phi-3-mini-4k-instruct"

# device_map="auto" distribuye el modelo en GPU/CPU automáticamente
model = AutoModelForCausalLM.from_pretrained(
```

```

        model_id,
        quantization_config=bnb_config,
        trust_remote_code=True,
        device_map="auto"
    )
    model.config.use_cache = False

tokenizer = AutoTokenizer.from_pretrained(model_id, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"

print("Modelo y tokenizador cargados.")
print("Dispositivo:", next(model.parameters()).device)

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/file_download.py:949: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. The equivalent of `resume_download` is now `continue_on_error`. The `continue_on_error` parameter will be `True` by default with the new default of `raise_other_exceptions=True`.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%                               967/967 [00:00<00:00, 86.8kB/s]

configuration_phi3.py:      11.2k/? [00:00<00:00, 867kB/s]
A new version of the following files was downloaded from https://huggingface.co/microsoft/Phi-3-mini-4k-instruct:
- configuration_phi3.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of the following files, you can add the following hashes to your trusted environment.
modeling_phi3.py:      73.2k/? [00:00<00:00, 6.59MB/s]
A new version of the following files was downloaded from https://huggingface.co/microsoft/Phi-3-mini-4k-instruct:
- modeling_phi3.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of the following files, you can add the following hashes to your trusted environment.
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.m
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.m
model.safetensors.index.json:      16.5k/? [00:00<00:00, 1.21MB/s]

Downloading shards: 100%                          2/2 [00:49<00:00, 23.40s/it]

model-00001-of-00002.safetensors: 100%            4.97G/4.97G [00:30<00:00, 191MB/s]

model-00002-of-00002.safetensors: 100%            2.67G/2.67G [00:18<00:00, 175MB/s]

Loading checkpoint shards: 100%                    2/2 [00:32<00:00, 15.36s/it]

generation_config.json: 100%                      181/181 [00:00<00:00, 22.6kB/s]

tokenizer_config.json:      3.44k/? [00:00<00:00, 269kB/s]

tokenizer.model: 100%                               500k/500k [00:00<00:00, 2.12MB/s]

tokenizer.json:      1.94M/? [00:00<00:00, 55.2MB/s]

added_tokens.json: 100%                             306/306 [00:00<00:00, 17.0kB/s]

special_tokens_map.json: 100%                       599/599 [00:00<00:00, 31.9kB/s]
Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned to the new vocabulary.
Modelo y tokenizador cargados.
Dispositivo: cuda:0

```

6. Crear el Dataset de Entrenamiento

El dataset debe tener una columna llamada `text` con las conversaciones formateadas usando el chat template del modelo.

Hay **dos opciones** según tu caso de uso:

Opción A – Dataset hardcodeado (rápido, para prueba)

Define ejemplos directamente en el notebook. Útil para verificar que todo funciona.

Opción B – Dataset desde archivo JSONL (recomendado para proyectos reales)

Carga conversaciones desde un archivo `.jsonl` que subas a Colab. Cada línea debe ser un JSON con la clave `messages` (lista de `{role, content}`).

Fix aplicado: El campo del dataset que recibe `SFTTrainer` debe llamarse `text` y contener la conversación ya formateada como string (usando `apply_chat_template`). La versión original pasaba la lista de mensajes directamente, lo que causaba errores.

```
from datasets import Dataset

# === OPCIÓN A: Dataset hardcodeado ===
# Modifica estos ejemplos con la personalidad/tarea que quieras entrenar.
# Cada entrada debe ser una conversación completa (puedes incluir múltiples turnos).

raw_data = [
    {"messages": [
        {"role": "user", "content": "Hoy he tenido un día terrible en el trabajo, todo salió mal."},
        {"role": "assistant", "content": "Recuerda que no controlamos los eventos externos, solo nuestro"}
    ]},
    {"messages": [
        {"role": "user", "content": "Estoy muy ansioso por una presentación importante que tengo mañana"},
        {"role": "assistant", "content": "La ansiedad a menudo proviene de temer un futuro que no ha ll"}
    ]},
    {"messages": [
        {"role": "user", "content": "Una persona me ha ofendido profundamente con sus palabras."},
        {"role": "assistant", "content": "Si te sientes herido por las palabras de otro, no son sus pal"}
    ]},
    {"messages": [
        {"role": "user", "content": "Siento que he perdido mucho tiempo en mi vida y me arrepiento."},
        {"role": "assistant", "content": "El pasado ya no existe y el futuro es incierto. Solo posees e"}
    ]},
]

# Función para formatear con el chat template del modelo
def format_chat(example):
    text = tokenizer.apply_chat_template(
        example["messages"],
        tokenize=False,
        add_generation_prompt=False
    )
    return {"text": text}

dataset = Dataset.from_list(raw_data)
dataset = dataset.map(format_chat, remove_columns=["messages"])

print(f"Dataset cargado con {len(dataset)} ejemplos.")
print("\n--- Ejemplo de dato formateado ---")
print(dataset[0]["text"])
```

Map: 100% 4/4 [00:00<00:00, 96.45 examples/s]

Dataset cargado con 4 ejemplos.

--- Ejemplo de dato formateado ---

<|user|>

Hoy he tenido un día terrible en el trabajo, todo salió mal.<|end|>

<|assistant|>

Recuerda que no controlamos los eventos externos, solo nuestra reacción ante ellos. ¿Qué aspecto de tu r

<|endoftext|>

```
# === OPCIÓN B: Dataset desde archivo JSONL ===
# Descomenta y adapta este bloque si usas un archivo .jsonl externo.
# El archivo debe estar subido a tu sesión de Colab (panel izquierdo > icono carpeta).
```

```
import json, os
from datasets import Dataset
```

```

FILE_PATH = "converted_dataset.jsonl" # <-- Cambia al nombre de tu archivo

if not os.path.exists(FILE_PATH):
    raise FileNotFoundError(f"No se encontró el archivo {FILE_PATH}. Súbelo a Colab.")

raw_data = []
with open(FILE_PATH, "r", encoding="utf-8") as f:
    for line in f:
        raw_data.append(json.loads(line))

def format_chat(example):
    text = tokenizer.apply_chat_template(
        example["messages"],
        tokenize=False,
        add_generation_prompt=False
    )
    return {"text": text}

dataset = Dataset.from_list(raw_data)
dataset = dataset.map(format_chat, remove_columns=["messages"])

print(f"Dataset cargado con {len(dataset)} ejemplos.")
print(dataset[0]["text"])

```

```

Map: 100%                               94/94 [00:00<00:00, 2180.63 examples/s]

Dataset cargado con 94 ejemplos.
<|user|>
¿Qué área del cerebro es crucial para la planificación, la toma de decisiones y el control de los impulsos?
<|assistant|>
La corteza prefrontal (PFC), responsable de las funciones ejecutivas como la planificación y el autocontrol.
<|endoftext|>

```

7. Probar el **Modelo Base** (antes del fine-tuning)

Genera una respuesta para comparar luego con el modelo afinado.

Fix aplicado: Se usa `.to("cuda")` directamente (más robusto con `device_map="auto"`) y la variable de entrada se llama `input_ids` de forma consistente.

```

prompt = "¿Qué papel juega el hipocampo en la navegación espacial?"
messages = [{"role": "user", "content": prompt}]

input_ids = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt"
).to("cuda")

with torch.no_grad():
    outputs = model.generate(
        input_ids,
        max_new_tokens=120,
        do_sample=True,
        temperature=0.7,
        top_k=50,
        top_p=0.95
    )

baseline_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
print("--- Respuesta del Modelo Base ---")
print(baseline_text.split("assistant\n")[-1])

```

de vuelo o las superficies de otros planetas. Aunque aún se está estudiando, se cree que la función del h

✓ 8. Configurar **LoRA** y el Entrenamiento (TRL - SFTTrainer)

Se aplican adaptadores LoRA a proyecciones clave/valor/consulta y capas MLP.

Fixes aplicados:

- `report_to="none"` en `TrainingArguments` evita el error de login a Weights & Biases.
- `dataset_text_field="text"` apunta a la columna correcta del dataset.
- El `trainer` se crea y entrena en **celdas separadas** para facilitar la depuración.

```
from peft import LoraConfig, get_peft_model
from transformers import TrainingArguments
from trl import SFTTrainer

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()

training_args = TrainingArguments(
    output_dir="./phi3-finetuned",
    per_device_train_batch_size=1,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    num_train_epochs=5,
    logging_steps=2,
    fp16=True,
    save_strategy="epoch",
    report_to="none", # FIX: evita el error de login a Weights & Biases
)

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
    dataset_text_field="text", # FIX: campo correcto generado en el paso 6
    max_seq_length=1024,
    tokenizer=tokenizer,
    packing=False,
)

print("Trainer configurado. Listo para entrenar.")

trainable params: 8,912,896 || all params: 3,829,992,448 || trainable%: 0.23271314815918928
Map: 100% 94/94 [00:00<00:00, 2294.92 examples/s]
Trainer configurado. Listo para entrenar.
/usr/local/lib/python3.12/dist-packages/accelerate/accelerator.py:479: FutureWarning: `torch.cuda.amp.GradScaler` is deprecated. Please use `torch.cuda.amp.grad_scaler.GradScaler` instead.
  self.scaler = torch.cuda.amp.GradScaler(**kwargs)

# Iniciar entrenamiento
trainer.train()
```

✓ 9. Probar el **Modelo Afinado** (después del fine-tuning)

Usa el **mismo prompt** del paso 7 para comparar estilos de forma justa.

Fix aplicado: Se eliminó `del trainer` que causaba un `NameError` si la celda se ejecutaba más de una vez. Sólo se limpia la caché de GPU con `empty_cache()`.

```
cuda.empty_cache()

inputs = [{"role": "user", "content": "Cual es la diferencia entre el cerebro corporal y el cerebro cognitivo"}]
tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt"
    cuda")

torch.no_grad():
    outputs = model.generate(
        input_ids,
        max_new_tokens=120,
        do_sample=True,
        temperature=0.7,
        top_k=50,
        top_p=0.95

    )

finetuned_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
"--- Respuesta del Modelo Afinado ---")
finetuned_text.split("assistant\n")[-1])
```

que el cerebro cognitivo (lóbulo temporal y elíptica) la procesa para formar una representación consciente

✓ 10. (Opcional) Guardar/Recargar Adaptadores LoRA

Guarda los pesos de LoRA para reusar sin reentrenar.

```
# Guardar adaptadores LoRA
save_dir = "./phi3-finetuned/adapters"
model.save_pretrained(save_dir)
tokenizer.save_pretrained(save_dir)
print(f"Adaptadores guardados en: {save_dir}")
```

Adaptadores guardados en: ./phi3-finetuned/adapters

```
# Ejemplo de recarga de adaptadores (en otra sesión)
# from peft import PeftModel
# base_model = AutoModelForCausalLM.from_pretrained(
#     model_id, quantization_config=bnb_config, trust_remote_code=True, device_map="auto"
# )
# tuned_model = PeftModel.from_pretrained(base_model, save_dir)
# tuned_model.eval()
# print("Adaptadores LoRA recargados.")
```

11. Análisis de Resultados (para entregar)

Responde aquí en celdas Markdown:

1. **Comparación cualitativa:** Copia/pega respuestas del **modelo base** y **finetune**. Describe diferencias en **tono**, **contenido** y **estilo**. ¿Fue exitoso el fine-tuning? ¿Por qué?
2. **Pérdida (loss):** Observa el *log* de entrenamiento. ¿Cómo evolucionó el `loss` por época? ¿Qué implica?
3. **Experimentación (opcional):** ¿Qué pasaría si:
 - a) duplicas ejemplos del dataset,
 - b) entrenas 10 épocas,
 - c) introduces ejemplos contradictorios?

4. **Aplicaciones:** Propón otra personalidad/tarea (p. ej. *experto culinario*, *haikus*, *tutor de física*, etc.) y diseña el dataset mínimo que necesitarías.

12. Entrega

- Enlace a tu **Colab** con celdas ejecutadas y salidas visibles.
- Documento con respuestas a **Análisis de Resultados**.

Nota: Revisa que tu entorno use **GPU T4** para que el entrenamiento sea viable en el tiempo estimado.